HW6: Design D1

Caden O'Leary, Elmoataz Abdellah, Matthew Goldsberry, Seunghun Lee, Kristen Griessman

Header, scope, and conventions

Project Title:

HABSat-1 On-Board Computer (OBC) Flight Software

Goal Statement:

In collaboration with the CubeCats organization, this project focuses on the design, implementation, and verification of the initial prototype flight software for the On-Board Computer (OBC) on the HABSat-1 mission. The flight software shall establish core Command and Data Handling (CDH) capabilities across satellite subsystems and the ground station. It will also be able to execute autonomous and scheduled tasks across mission flight states.

Scope:

This document details the data structures and component interfaces for the Main Computer, ACDS Computer, Power Computer, Imager Adapter, Data Processor, and Data Storage, while the specific implementations for Telemetry/Command Radio, Main Downlink Radio, and Ground Station data pipeline are deferred to Design D2.

Diagram Conventions:

Boxes:

Represent distinct hardware entities.

Rows within boxes:

Represent the key attributes, state variables, telemetry data (e.g., temp, voltage), or memory buffers maintained by that specific component.

Lines:

Represent direct data flow or physical communication bus interfaces between components.

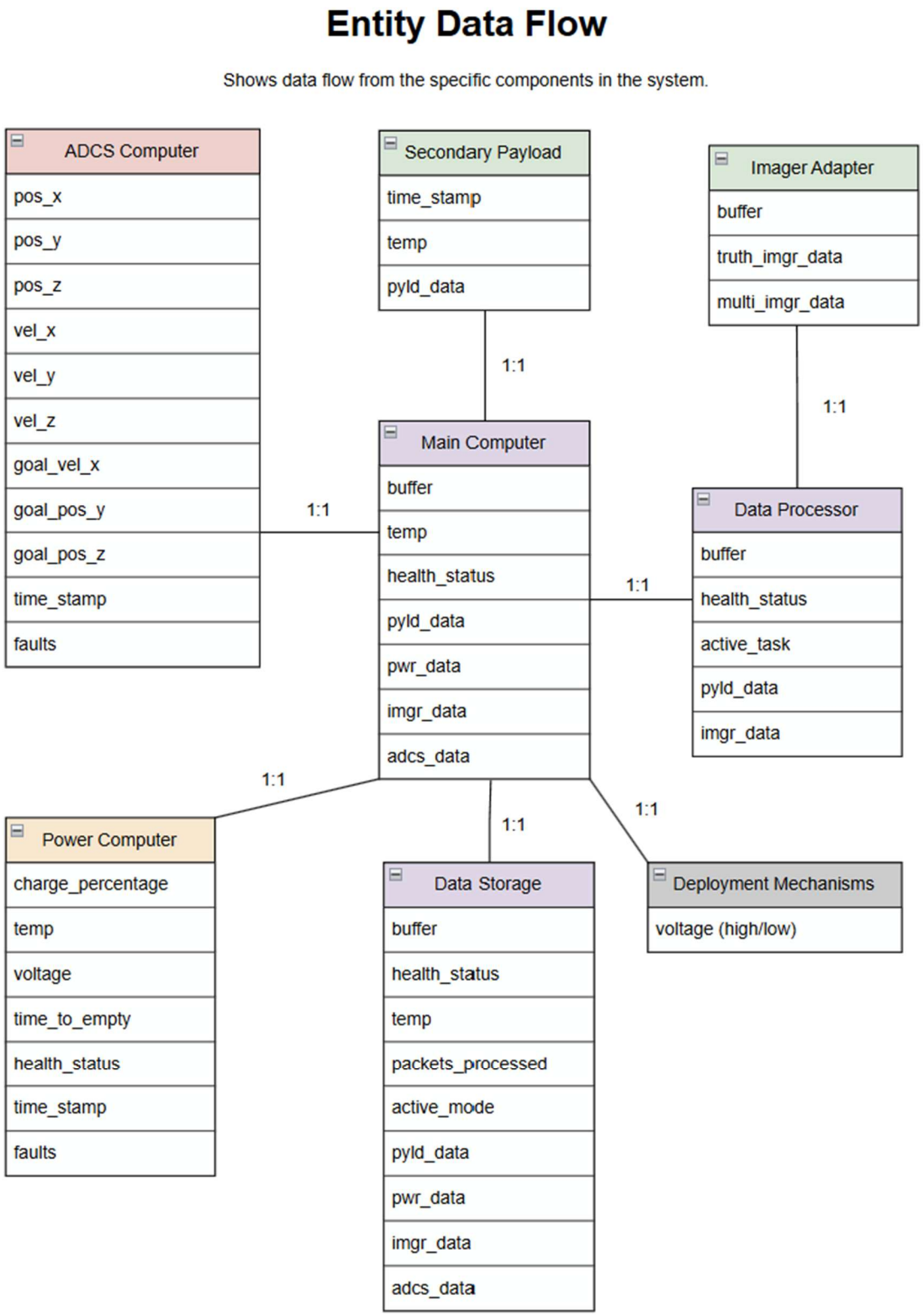
Keys:

This schema models embedded hardware states rather than a traditional relational database. Because of this, there are no primary and foreign keys. data is uniquely identified by the hardware component that generates it.

Cardinality:

Relationships are drawn as plain, un-forked lines representing strict 1:1 cardinality, as the connections exist strictly between singular, point-to-point physical hardware components.

Data Model


Entity Data Flow
Shows data flow from the specific components in the system.
ADCS Computer
pos_x
pos_y
pos_z
vel_x
vel_y
vel_z
goal_vel_x
goal_pos_y
goal_pos_z
time_stamp
faults
Secondary Payload
time_stamp
temp
pyld_data
Imager Adapter
buffer
truth_imgr_data
multi_imgr_data
Main Computer
buffer
temp
health_status
pyld_data
pwr_data
imgr_data
adcs_data
Data Processor
buffer
health_status
active_task
pyld_data
imgr_data
Power Computer
charge_percentage
temp
voltage
time_to_empty
health_status
time_stamp
faults
Data Storage
buffer
health_status
temp
packets_processed
active_mode
pyld_data
pwr_data
imgr_data
adcs_data
Deployment Mechanisms
voltage (high/low)
1:1
1:1
1:1
1:1
1:1
1:1

Relational Model vs Simpler Store

For a successful RTOS, this whole connection needs to be a simpler store model. We chose a simpler store rather than a relational model because a relational database introduces significant processing overhead, dynamic memory usage, and unpredictable latency that would violate the hard real-time deadlines of the main mission mode loops.

Reasons for entities

Entity	Reason
ADCS Computer	The ACDS computer is the central logic for ADCS Subsystem mission
Power Computer	The power computer is the central logic for Power Subsystem mission
Payload	The payload is currently undecided for its data, but multiple attributes need to be considered that are part of the payload
Imager Adapter	The imager adapter will communicate multiple attributes with other entities, which requires it to be an entity
Data Storage	The storage is a critical component that holds critical attributes of the Main Computer
Data Processor	The data processor is a critical component that holds critical attributes of the Main Computer
Deployment Mechanisms	The deployment mechanisms are a critical component in the mission
Main Computer	The main mission operation for this project

Reasons for cardinality

All the relationships in this diagram are a 1:1 cardinality, as the connections are between singular hardware components.

Core Algorithms

Pass Execution Algorithm Specification

What Problem this Solves

Pass execution is a deterministic, event-driven sequencer running on the Main Computer. It orchestrates subsystem hardware components during brief orbital imaging windows.

This algorithm coordinates power switching, camera sensor pre-warming, ADCS closed-loop target tracking, image capture, and return to minimal-drag orientation. Without deterministic sequencing, minor timing lags cause the imager to miss the targets. Unmonitored failures risk unnecessary battery depletion, sensor overheating, and/or excessive drag.

Inputs and Outputs

Inputs (values):

- Current spacecraft epoch time in seconds (uint32_t)
- Pass Record from the schedule queue (struct):
    - Unique pass sequence number (uint16_t)
    - Target acquisition start timestamp, in epoch seconds, for target (uint32_t)
    - Target acquisition end timestamp, in epoch seconds, for target (uint32_t)
    - Target Latitude (float32)
    - Target Longitude (float32)
    - Target Altitude (float32)
- Static timing thresholds (struct)
    - Lead slew seconds for ADCS alignment (uint8_t)
    - Lead warmup seconds for cameras (uint8_t)
    - Max time for the ADCS to acquire the lock (uint8_t)
    - Min State of Charge (uint8_t)

Inputs (Commands):

- Boresight aligned with target (MSG_ADCS_TARGET_ACQUIRED)
- Camera warm-up done (MSG_IMG_WARMUP_READY)
- Capture executed (MSG_IMG_CAPTURE_COMPLETE)
- Battery State of Charge (MSG_PWR_VITALS_DATA)
- Low power alert (MSG_PWR_LOW_BATTERY_ALERT)
- Imager fault (MSG_IMG_HARDWARE_FAULT)

Outputs:

- Execution outcome (enum)
- Captured Frames (uint4_t)
- Next state mode transition (enum)
- Outgoing CAN command frames (CMD_ADCS_SET_MODE, CMD_PWR_RAIL_ON/OFF, CMD_IMG_INIT_WARMUP, CMD_IMG_TRIGGER_BURST, CMD_ADCS_STOP_TRACK)

Expected Complexity

There will be one active pass evaluated per pass and a schedule buffer of $N <= 16$ entries.

- **Time Complexity**: This will be O(1) worst-case execution since all actions are linear and constant.
- **Space Complexity:** O(1) all information in this pass (barring the new images taken) is statically defined in the scope of this execution; there is no dynamic memory being allocated during this.

Alternatives Considered

- **Alternative 1:** Direct ADCS to Imager triggering. This was rejected because it would either bypass the centralized checks that the Main Computer operates. It would require the ADCS also having access to the system telemetry of the whole satellite to be able to perform the required safety checks before imaging, while also continuously reading large amounts of sensor inputs and having to command its actuators to orient and stay on target.
- **Alternative 2:** Real-time orbit propagation on the OBC. This was rejected because orbital math is computationally expensive and belongs on its own isolated computer, and not on the Main Computer where it would likely cause scheduling latency.

Edge Case Handling

- **Camera Link / Warmup Failure:** If MSG_IMG_HARDWARE_FAULT is asserted or warmup exceeds 60 seconds, camera power is killed, tracking is stopped, and the pass terminates with ABORT_FAULT.
- **In-Pass Low Battery Trip**: If MSG_PWR_LOW_BATTERY_ALERT triggers or battery State of Charge is less than the defined threshold, the pass aborts immediately: camera rails are cut (CMD_PWR_RAIL_OFF), ADCS is commanded to orient towards the Sun, and the OBC transitions to Low Power.

- **ADCS Delay / Pointing Lock Timeout:** If the ADCS fails to assert MSG_ADCS_TARGET_ACQUIRED within the defined threshold time or passes the defined end timestamp for the pass, the sequencer aborts: cuts camera power, issues CMD_ADCS_STOP_TRACK, sets ADCS to Minimal Drag, and transitions to Standby. No images are taken.
- **Empty Input / No Passes**: If the pass queue is empty, the sequencer remains idle, commands ADCS to Minimal Drag along the ram vector, and keeps imaging rails off.
- **Duplicate Passes:** If a candidate pass shares an identical sequence ID or target window with an existing entry, the duplicate is dropped during queue ingestion to prevent redundant slewing.
- **Ties / Overlapping Passes:** If two passes contend for the same time window, the sequencer resolves the tie by selecting the pass with higher science priority (or higher maximum elevation), skipping the lower-ranked pass.
- **Ground Comms Preemption:** If RADIO_CARRIER_DETECT triggers during orientation or tracking, the pass aborts immediately, camera rails are cut, and ADCS transitions to Ground Station Pointing.
- **Clock Step / Time Sync Jump:** If a ground time synchronization shifts spacecraft epoch time mid-pass, phase transitions evaluate monotonic tick counters rather than raw wall-clock subtraction to prevent false timeouts.

Build-vs-Reuse Decisions

Component	Decision	Library / Service	License	Reason
Core Flight Software Framework	Reuse	NASA F' (F Prime)	Apache 2.0	Highly mature JPL-backed framework offering permissive open-source licensing, statically allocated memory for real-time performance, and a perfect fit for resource-constrained CubeSats.
Real-Time OS Kernel (OBC)	Reuse	FreeRTOS	MIT	Industry-standard microkernel with decades of maturity providing sub-

				microsecond determinism for our critical ADCS tasks, completely free to use, and natively fitting our chosen ARM Cortex-M hardware.
ADCS Attitude & Torque Control	Build	C++ Module	N/A	Built in-house because the magnetorquer equations and sun sensor filtering logic are unique to our physical hardware geometry.
Payload Chlorophyll Index Processor	Build	C++ Module	N/A	Built in-house because our specific multispectral band calculation is the primary science deliverable of the mission and has no off-the-shelf library equivalent.
Hardware Health & Watchdog	Reuse, then wrap	F' Svc::Health	Apache 2.0	We will wrap this highly mature, flight-proven JPL logic because it perfectly fits our watchdog requirements while maintaining strict execution performance.

API Contract

1. Interface Protocols & Framing Architecture

The HABSat-1 Main Computer (On-Board Computer / OBC) orchestrates satellite operations across four physical communication interfaces. Messages across these interfaces are classified into four operational categories:

- **Command Request (OBC → Subsystem):** An instruction sent by the OBC directing a subsystem to change state, execute a task, or configure operational parameters.
- **Command Response / ACK (Subsystem → OBC):** Immediate acknowledgment verifying whether a command frame was accepted, rejected, or malformed.
- **Telemetry Pull (OBC ↔ Subsystem):** Request-response exchange returning sampled sensor or status data to the OBC.
- **Asynchronous Event / Push (Subsystem → OBC):** Autonomous notifications triggered by external physical events, state completions, or threshold trips that drive OBC flight mode state transitions.

Frame Structures

CAN Bus (12 Bytes Fixed):

CAN STR	CAN ID	CAN Payload	CAN CRC	CAN ETX
1 Byte	1 Byte	8 Bytes	1 Byte	1 Byte

UART (10 + N Bytes):

Preamble	NodeID	FrameID	Payload	Checksum	ETX
1 Byte	1 Byte	1 Byte	Dynamic N Bytes	1 Byte	1 Byte

SPI (4-Wire Master/Slave):

Chip Select (SS)	SCLK	MOSI	MISO
Register & address selection	Bus clock	Command/data in	Status/data out

GPIO (Discrete Lines):

PIN ID	Data State
Hardware pin signaling line	0 (High to Low) / 1 (Low to High)

2. Subsystem Command & Telemetry APIs

2.1 ADCS Computer (Protocol: CAN Bus)

Controls spacecraft attitude determination, reaction wheel actuators, orbit propagation, and target pointing.

Identifier	Message Name	Direction	Message Type	Payload Structure (64-bit)	Functional Description
0x01	CMD_ADCS_GET_STATUS	OBC → ADCS	Telemetry Req	0x00000000	Polls system health.
0x02	CMD_ADCS_SET_MODE	OBC → ADCS	Command Req	0x01: Detumble, 0x02: Sun-Pointing, 0x03: Minimal Drag (Ram), 0x04: Target Tracking, 0x05: Ground Station Pointing	Commands active attitude orientation profile.
0x03	CMD_ADCS_LOAD_TLE	OBC → ADCS	Command Req	Buffer ID (8-bit), Chunk Index (56-bit)	Transfers ground-uplinked orbital elements.
0x04	CMD_ADCS_SET_TARGET_LAT	OBC → ADCS	Command Req	IEEE 754 Float: Latitude	Sets designated Earth target latitude.
0x05	CMD_ADCS_SET_TARGET_LON	OBC → ADCS	Command Req	IEEE 754 Float: Longitude	Sets designated Earth target longitude.

Identifier	Message Name	Direction	Message Type	Payload Structure (64-bit)	Functional Description
0x06	CMD_ADCS_SET_TARGET_ALT	OBC → ADCS	Command Req	IEEE 754 Float: Altitude	Sets designated Earth target altitude.
0x07	CMD_ADCS_REQ_SCHEDULE	OBC → ADCS	Telemetry Req	0x00000000	Requests pass windows generated by 30-min orbit propagation routine.
0x08	CMD_ADCS_STOP_TRACK	OBC → ADCS	Command Req	0x00000000	Ends closed-loop Earth tracking after exposure completion.
0x40	MSG_ADCS_ACK	ADCS → OBC	Cmd Response	Cmd CAN ID (8-bit) Status (8-bit) Err Code (16-bit)	Sent synchronously upon receipt of any ADCS command (0x00 = ACK).
0x41	MSG_ADCS_DETUMBLE_COMPLETE	ADCS → OBC	Async Event	0x00000000	Body angular rates fall below threshold; OBC sets DETUMBLE_COMPLETE_FLAG = TRUE.
0x42	MSG_ADCS_TARGET_ACQUIRED	ADCS → OBC	Async Event	0x00000000	Closed loop tracking locked within target tolerance; triggers payload exposure.
0x43	MSG_ADCS_SCHEDULE_PACKET	ADCS → OBC	Telemetry Resp	Pass ID (8-bit) Timestamp (24-bit)	Sequence of validated target imaging windows to arm OBC event queue.

Identifier	Message Name	Direction	Message Type	Payload Structure (64-bit)	Functional Description
0x44	MSG_ADCS_POINTING_TIMEOUT	ADCS → OBC	Async Alert	0x000000000	Triggered when closed-loop target tracking fails to acquire the target before the scheduled window expires.
0x45	MSG_ADCS_STATUS_DATA	ADCS → OBC	Telemetry Resp	Sensor validity flags	Response payload to CMD_ADCS_GET_STATUS.

2.2 Power Computer (Protocol: CAN Bus)

Monitors battery health, bus rails, charging states, and watchdog failsafe.

Identifier	Message Name	Direction	Message Type	Payload Structure (64-bit)	Functional Description
0x20	CMD_PWR_GET_VITALS	OBC → EPS	Telemetry Req	0x00000000	Queries battery voltages, currents, temperatures, and State-of-Charge.
0x21	CMD_PWR_RAIL_ON	OBC → EPS	Command Req	(Rails: 0x00 Imagers, 0x01 Data Processor, 0x02 Main Downlink Radio)	Enables power to specific payload rails.
0x22	CMD_PWR_RAIL_OFF	OBC → EPS	Command Req	(Rails: 0x00 Imager, 0x01 Data Processor, 0x02 Main	Disables power to specific payload rails.

Identifier	Message Name	Direction	Message Type	Payload Structure (64-bit)	Functional Description
				Downlink Radio)	
0x23	CMD_PWR_GET_CHARGE_STATE	OBC → EPS	Telemetry Req	0x00000000	Queries solar panel charging current generation.
0x50	MSG_PWR_ACK	EPS → OBC	Cmd Response	CAN ID (8-bit), Rail (8-bit), Status (16-bit)	Verifies rail command execution.
0x51	MSG_PWR_LOW_BATTERY_ALERT	EPS → OBC	Async Alert	Battery SoC (%)	Signals a critical battery threshold trip.
0x52	MSG_PWR_CHARGE_RECOVERED	EPS → OBC	Async Event	Battery SoC (%)	EPS verifies batteries recharged above operational thresholds.
0x53	MSG_PWR_VITALS_DATA	EPS → OBC	Telemetry Resp	Battery Voltage, Bus Current, Battery Temp, SoC	Response payload to CMD_PWR_GET_VITALS.
0x54	MSG_PWR_DEPLOY_CONFIRM	EPS → OBC	Telemetry Resp	0x00000000	Verifies solar array current generation following deployment actuation.

2.3 Image Adapter (Protocol: CAN Bus)

Orchestrates the Truth Imager and Multispectral Imager. (Note: High-rate raw image frames stream directly to the Data Processor via UART).

Identifier	Message Name	Direction	Message Type	Payload Structure (64-bit)	Functional Description
0x30	CMD_IMG_INIT_WARMUP	OBC → Imager	Command Req	Truth 0x01, Multispectral 0x02	Pre-warms camera sensors prior to target acquisition.
0x31	CMD_IMG_TRIGGER_BURST	OBC → Imager	Command Req	Camera: 0x01 (Truth), 0x02 (Multispectral), Frame Count (8-bit), Exposure Duration (16-bit)	Triggers synchronized exposure sequence.
0x32	CMD_IMG_GET_STATUS	OBC → Imager	Telemetry Req	0x00000000	Polls capture and sensor buffer status.
0x33	CMD_IMG_PURGE_BUFFER	OBC → Imager	Command Req	Target Image ID	Purges imager buffer once storage write is verified.
0x60	MSG_IMG_ACK	Imager → OBC	Cmd Response	CAN ID (8-bit), Camera ID (8-bit), Status (16-bit)	Confirms warmup, capture trigger, or purge execution.
0x61	MSG_IMG_WARMUP_READY	Imager → OBC	Async Event	CAN ID (8-bit), Temp (32-bit)	Notifies OBC that camera sensors reached operating temperatures.
0x62	MSG_IMG_CAPTURE_COMPLETE	Imager → OBC	Async Event	Total frames successfully	Informs OBC that capture burst finished.

Identifier	Message Name	Direction	Message Type	Payload Structure (64-bit)	Functional Description
				captured and buffered	
0x63	MSG_IMG_HARDWARE_FAULT	Imager → OBC	Async Alert	Error Code (0x01: Timeout, 0x02: Link Lost)	Hardware or Camera Link failure alert.

2.4 Data Processor (Protocol: CAN Bus)

Controls execution of payload processing algorithms and S-band downlink routing.

Identifier	Message Name	Direction	Message Type	Payload Structure (64-bit)	Functional Description
0x10	CMD_DP_START_PROCESSING	OBC → DP	Command Req	Pipeline ID (0x01: Chlorophyll Index Conversion)	Starts image loading from the imager buffer and index calculation.
0x11	CMD_DP_ABORT	OBC → DP	Command Req	Abort Reason (0x01: Low Power, 0x02: Comms Preempt)	Halts processing pipelines immediately.
0x12	CMD_DP_TRIGGER_DOWNLINK	OBC → DP	Command Req	Target Image File ID or Transmission Mask	Commands DP to stream payload files directly over S-Band.
0x70	MSG_DP_ACK	DP → OBC	Cmd Response	CAN ID (8-bit), Status (8-bit), Error Code (16-bit)	Acknowledges processing, abort, or downlink triggers.

Identifier	Message Name	Direction	Message Type	Payload Structure (64-bit)	Functional Description
0x72	MSG_DP_PROCESSING_DONE	DP → OBC	Async Event	Count of generated chlorophyll index images	Confirms image pipeline completion and storage offload.
0x73	MSG_DP_DOWNLINK_STATUS	DP → OBC	Telemetry Resp	Bytes Transmitted (24-bit), Status (8-bit)	Relays real-time S-band transmission progress to OBC.

2.5 Telemetry / Command Radio - UHF (Protocol: UART)

Manages ground station telecommand uplink, housekeeping downlink, and amateur digipeater frames.

Frame ID	Frame / Command Name	Direction	Message Type	Payload Structure (Fixed 10+N Bytes)	Functional Description
0xA1	RADIO_BEACON_TX	OBC → Radio	Command Req	Battery mV/SoC, temperatures, boot count, mission flags	Transmits a 49-second unencrypted housekeeping beacon.
0xA2	RADIO_SEND_DOWNLINK	OBC → Radio	Command Req	Real-time health logs, system events, command ACKs	Transmits omnidirectional telemetry frames during ground pass.
0xA3	RADIO_DIGIPEAT_STAGING	OBC → Radio	Command Req	Re-encoded digipeater amateur payload	Stages verified amateur frames for authorized downlink transmission.
xxxx	PACKETS	Radio → OBC	Data Stream	TBD in D2	Packets from GS or Amateur radio signals

2.6 Data Storage (Protocol: SPI)

Non-volatile memory storage (NVM) for mission state flags, logs, and image indexes.

Register	Operation Name	Direction	Message Type	Data / Parameters	Functional Description
0x0010	DS_READ_FLAGS	OBC ↔ DS	Telemetry Pull	MOSI: 0x0010 → MISO: 3-byte bitmask (SOLAR_DEPLOY, DETUMBLE, COMMISSIONED)	Evaluates persistent mission progress flags to determine mode.
0x0010	DS_WRITE_FLAG	OBC → DS	Command Req	MOSI: Register (16-bit), Flag ID (8-bit), Value (8-bit)	Writes mission phase updates to persistent NVM.
0x0100	DS_WRITE_LOG	OBC → DS	Command Req	MOSI: Register (16-bit), Len (8-bit), Log Data	Records time-tagged housekeeping metrics and fault events.
0x0200	DS_CHECK_IMAGE_INDEX	OBC ↔ DS	Telemetry Pull	MOSI: 0x0200 → MISO: 0x01 (Stored) / 0x00 (Pending)	Verifies chlorophyll images uploaded by DP before cutting power.

2.7 Deployment Mechanisms (Protocol: GPIO)

Discrete hardware actuation lines for physically releasing the solar array.

Pin Name	Direction	Interface Type	Electrical Value	Functional Description
PIN_DEP_SOLAR_ACT	OBC → Deployable	GPIO Output	Pulse High (1 then 0)	Actuates solar panel release mechanism.
PIN_DEP_CAMERA_LENS	OBC → Deployable	GPIO Output	Pulse High (1 then 0)	Actuates camera lens release mechanism.

Pin Name	Direction	Interface Type	Electrical Value	Functional Description
PIN_DEP_ANTENNAS	OBC → Deployable	GPIO Output	Pulse High (1 then 0)	Actuates antennas deployment mechanism.

2.8 Secondary Payload (Protocol: UART)

Standardized secondary mission communication payload interface. TBR once secondary payload is chosen.

Frame ID	Frame / Command Name	Direction	Message Type	Payload Structure (Fixed 10+N Bytes)	Flight Mode & Functional Description
0xB1	SEC_PAY_PING	OBC → Payload	Command Req	Keep-alive check byte (0x00)	Verifies interface responsiveness during Power-On Self-Test.
0xB2	SEC_PAY_START_TASK	OBC → Payload	Command Req	Task configuration ID	Triggers secondary experiment sequence.
0xB3	SEC_PAY_READ_DATA	OBC → Payload	Telemetry Req	Target data block ID	Polls experiment data for storage and downlink.
0xD0	SEC_PAY_FRAME_ACK	Payload → OBC	Cmd Response	TX Frame ID (8-bit), ACK/NACK (8-bit)	Confirms secondary payload frame receipt.
0xD1	SEC_PAY_DATA_PACKET	Payload → OBC	Telemetry Resp	Dynamic data payload block	Science metrics returned to OBC.

Technology Choices & Explanations

Framework: NASA F' (F Prime)

We have selected F' as the primary flight software framework for the HABSat-1 On-Board Computer. F' provides an optimal combination of features that satisfy our driving requirements while scoring the highest across our evaluation criteria.

Team Skill Fit:

F' utilizes modern C++ for flight code and Python for its Ground Data System (GDS) tooling. This aligns perfectly with the current programming skillsets of the club members, allowing for rapid development. Alternatives like NASA cFS were not chosen due to their steep learning curve and dense, legacy C-based architecture.

Licensing:

F' is licensed under the Apache 2.0 open-source license, which allows the club to freely develop, modify, and distribute the software without the legal or intellectual property restrictions associated with proprietary systems.

Community Support:

F' is maintained by NASA JPL and possesses a highly active open-source GitHub community, extensive documentation, and numerous CubeSat missions.

Performance:

F' uses a component-driven architecture with static memory allocation and strongly typed message-passing ports. This guarantees the deterministic, real-time performance needed to safely execute ADCS control loops and route high-throughput multispectral imagery.

Cost and Hosting:

The framework is entirely free to use ($0 cost) and is heavily optimized for resource-constrained embedded platforms. It integrates with free RTOS environments on our chosen hardware, eliminating the licensing and hosting fees required by commercial options like the VxWorks framework.

Hardware: EnduroSat OBC Type I Board

We selected the EnduroSat OBC Type I over the Raspberry Pi Compute Module 5 because it provides a fully integrated, space-qualified embedded platform that directly meets our real-time satellite requirements without necessitating complex custom carrier boards.

Team Skill Fit:

The EnduroSat OBC is based on an ARM Cortex M4 or M7 microcontroller, which perfectly aligns with the club's embedded systems expertise and our prior selection of the C++-based NASA F' flight software framework. In contrast, the Raspberry Pi CM5 requires managing a full Linux OS environment, which significantly complicates real-time task scheduling and execution.

Licensing:

The hardware allows us to compile, flash, and execute our own software stacks using open-source RTOS environments (like FreeRTOS or Zephyr) and frameworks without having to pay proprietary software license fees or being forced into restrictive vendor-locked toolchains.

Community Support:

While the Raspberry Pi ecosystem has a massive general-purpose community, EnduroSat provides dedicated aerospace engineering support and proven spaceflight missions. They are backed by ECSS and GEVS standard space qualification testing campaigns (including thermal vacuum and vibration testing).

Performance:

Although the CM5 possesses a much faster 2.4 GHz processor, the EnduroSat OBC is purpose-built for ultra-low power satellite operations, consuming only 104-123 mA at 180 MHz. Furthermore, the EnduroSat board natively integrates dual-redundant 3-axis accelerometers and digital compasses, alongside hardware PWM outputs for magnetorquer control, providing deterministic ADCS hardware capabilities that the CM5 lacks out of the box.

Cost and Hosting:

While the EnduroSat board is significantly more expensive at retail than the Raspberry Pi CM5, we already have the EnduroSat board on hand from previous club funding, bringing the actual project cost to $0. Additionally, the board conforms to the standard CubeSat PC/104 stack physical form factor, eliminating the need to design and manufacture a custom carrier board.